



TECHNICAL PAPER 005

CA-IDMS & ADS/Online Modernization

From Network Databases and Legacy Dialogs to Relational Cloud Architecture

CA-IDMS • ADS/Online • COBOL • CICS • Java • Amazon Aurora PostgreSQL • Discovery • Validation

BeyondX Engineering Position

Modernization succeeds when business relationships and behavior are preserved - not when legacy structures are merely translated into newer syntax.

Version 6.0 | Final Publish Edition | September 2026

Technology with Purpose. Opportunity with Impact.

Executive Summary

CA-IDMS and ADS/Online modernization is fundamentally different from a simple database migration. CA-IDMS represents relationships through a network model in which records participate in named sets



and applications navigate owner/member relationships. ADS/Online adds a tightly coupled application layer of dialogs, processes, maps/screens, business rules and IDMS DML. An enterprise may therefore need to modernize both the database and the online application environment as interconnected workstreams.

The engineering challenge is not merely to convert records into tables or dialogs into Java classes. It is to discover the business relationships encoded in schemas, subschemas, owner/member sets, DBKEYs and COBOL layouts, while also tracing how ADS/Online dialogs and COBOL programs navigate those structures. The target must preserve business behavior while replacing pointer-driven navigation and proprietary runtime dependencies with explicit relational relationships and modern services.

Core Principle

The unit of modernization is not the IDMS record. The true unit of modernization is the business relationship.

This paper presents three legitimate modernization models. A phased mainframe path first removes CA-IDMS and ADS/Online dependencies while retaining the mainframe - for example, CA-IDMS to Db2 z/OS and ADS/Online to COBOL/CICS with embedded SQL - and later moves the relational application to cloud. An application-first hybrid path moves CA-IDMS to Db2 z/OS while transforming ADS/Online and selected COBOL logic to Java/Spring, establishing a relational application boundary that can substantially reduce application change when Db2 is later migrated to Amazon Aurora PostgreSQL. A direct path transforms CA-IDMS to Aurora PostgreSQL and ADS/COBOL application logic to Java/Spring on AWS. None of these models is mandatory; the choice should follow risk, business objectives, application dependencies, operational constraints and decommissioning goals.

1. Why CA-IDMS Modernization Is Different

Relational modernization begins with a change in data-access philosophy. A CA-IDMS application typically reaches related records by navigating a predefined path. A relational application describes the desired result set and allows the database optimizer to determine an access plan.

CA-IDMS NAVIGATIONAL ACCESS	RELATIONAL / SQL ACCESS
Find an owner record	Identify a business key or predicate
Navigate a named set	Join related tables
OBTAIN FIRST / NEXT	SELECT the required result set
Follow set links and maintain currency	Use PK/FK relationships, indexes and transactions
Procedural path determines access	Optimizer chooses the physical access plan



BeyondX Principle

Do not reproduce pointer architecture in relational syntax. Preserve the business relationship, ordering requirement, integrity rule and access behavior that the pointer structure was designed to support.

2. Inside the CA-IDMS Network Model

A disciplined migration starts by rebuilding an accurate model of the source. Important artifacts include the schema and subschemas, database areas, record types and elements, set definitions, access modes, indexes, DBKEY usage, application DML and production data relationships.

Legacy Concept	Modernization Interpretation
Schema	Enterprise database definition and relationship inventory
Subschema	Application-specific view of records and sets
Area	Physical/logical storage grouping requiring placement and volume analysis
Record type	Candidate relational entity/table; not always one-to-one
Set	Named owner/member relationship plus access path
Owner / member	Relationship roles requiring cardinality and optionality analysis
DBKEY	Legacy physical identity useful for migration traceability
CALC / index	Keyed access pattern and candidate relational key/index
Currency	Navigational processing state that normally disappears in set-oriented SQL

2.1 Sets, Owners, Members and Pointers

Figure 2. Simplified owner/member navigation pattern.

A set can carry more than referential meaning. It can encode membership rules, ordering, optionality and an expected access path. Those characteristics must be discovered before deciding whether the relational target is a foreign key, bridge table, child table, ordering column, index, constraint, or a combination of these.



3. From Network Navigation to Relational Relationships

A useful first-pass mapping is shown below. It is a discovery aid, not a mechanical conversion specification.

CA-IDMS Construct	Candidate Relational Representation	Engineering Question
Owner-member set	PK/FK relationship	Is the relationship 1:N, 1:1, optional or polymorphic?
Intersection record	Associative/bridge table	Does it carry attributes beyond the relationship?
Sorted set	FK + ordering column/index	Is order part of business semantics or only access optimization?
CALC key	PK, unique key or index	Is it stable and business-meaningful?
DBKEY	Temporary legacy cross-reference	Is any application logic incorrectly dependent on physical identity?
Multi-set membership	Multiple relationships / bridge model	What business role does each set represent?
OCCURS	Child table or explicit columns	Is repetition a business entity, fixed structure or presentation artifact?
REDEFINES	Subtype/detail tables or explicit model	What discriminator controls interpretation?

3.1 Navigational DML to SQL

Consider an application that locates a member and walks all claims within a set. The goal is not to translate each navigation verb into a separate database call. The goal is to identify the result set the business process requires.

Performance Implication

The IDMS set was both a relationship and an access path. When the relationship becomes a PostgreSQL foreign key, the migration must separately design the indexes needed for the dominant access patterns.



4. Complex COBOL Structures

4.1 OCCURS

Repeated COBOL structures should not automatically become numbered columns. When each occurrence represents a repeatable business entity, normalization into a child table usually produces a clearer relational model.

4.2 REDEFINES

REDEFINES can allow the same bytes to have different logical meanings. Migration requires discovery of the discriminator and business rules that determine which interpretation applies. The target may use subtype/detail tables, explicit nullable columns, or another deliberate relational representation. JSONB is useful for genuinely semi-structured data, but it should not become a default escape hatch for difficult legacy structures.

BeyondX Principle

Use PostgreSQL JSONB because the business data is naturally semi-structured - not merely because the source structure is difficult to normalize.

5. Date and Time Semantics: Storage, Access and Migration

CA-IDMS estates can contain more than one generation of date/time representation. IDMS SQL supports native date/time data types, while legacy CODASYL network records frequently carry dates and times in application-defined numeric, packed-decimal, character, Julian/ordinal or work-field representations. A modernization team must therefore discover not only how a value is physically stored, but how applications interpret it.

The critical distinction is semantic: a field that looks like YYYYMMDD may represent a calendar date, while a timestamp-looking value may represent local business time, server time, UTC, or a timezone-independent business event. The target type should follow the business meaning - not merely the source byte layout.

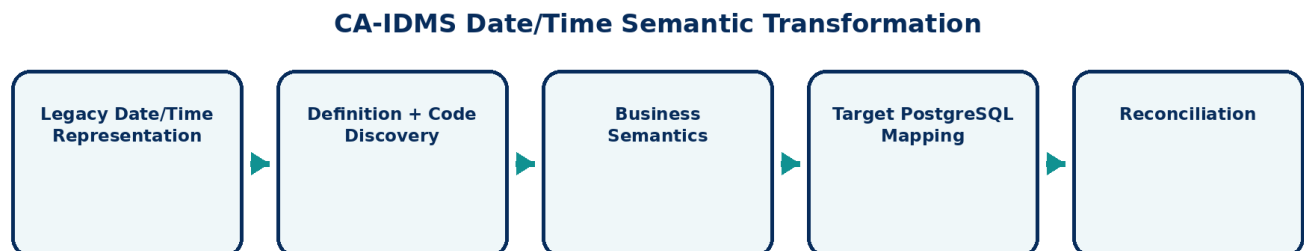


Figure - Date/time modernization must move from representation discovery to semantic mapping and reconciliation.



5.1 Storage and Access Patterns

For native IDMS SQL DATE, TIME and TIMESTAMP values, preserve the logical date/time semantics and supported precision when mapping to PostgreSQL. For legacy network records, inspect schema/subschema definitions together with COBOL PIC clauses, REDEFINES, ADS/Online logic, conversion routines, callable services and representative production values. Built-in or system date/time services can be important evidence of how an application converts, displays or generates a value, but the migration should not assume that every record element uses the SQL-native representation.

Legacy representation	Discovery question	Typical target direction
YYMMDD numeric / packed	What century or windowing rule applies? Are zero or sentinel values valid?	DATE after validated conversion
YYYYMMDD numeric / character	Are blanks, zeros, invalid calendar values or sentinels meaningful?	DATE after cleansing / rule preservation
Julian / ordinal encoding	Which convention and epoch are used by the application?	DATE after explicit conversion
IDMS SQL DATE / TIME	What logical precision and business semantics must be preserved?	PostgreSQL DATE / TIME as appropriate
Timestamp-like value	Is it local time, server time, UTC, or timezone-independent?	TIMESTAMP or TIMESTAMPTZ according to semantics
Application work field	How is it populated, formatted, compared and persisted?	Derive target type from business use

5.2 Timezone, Precision and Sentinel Semantics

Do not automatically attach UTC simply because the target platform is cloud-based. A value such as 2026-09-26 14:30:00 does not reveal its timezone by itself. Discovery should determine the generating system, business location, daylight-saving behavior, precision, comparison rules and whether the value represents an instant in time or a wall-clock business value. Sentinel values such as all zeros, high dates, low dates, blanks or application-specific "unknown" dates must also be mapped deliberately rather than silently coerced.

BeyondX Principle: Do not migrate the representation of a legacy date/time value until you have discovered its business meaning, precision, timezone semantics and conversion rules.

5.3 AI-Assisted Date/Time Discovery and Validation

ChatGPT and other AI engineering agents can help correlate record definitions, COBOL PIC clauses, REDEFINES, ADS/Online expressions, SQL, date conversion routines and test data to propose candidate mappings. This is particularly useful for identifying two-digit-year rules, duplicated conversion logic and



inconsistent representations across online and batch workloads. AI-generated mappings remain hypotheses until validated by engineers against authoritative code paths, production data and business rules.

Date/time reconciliation should explicitly test nulls and blanks, zero and sentinel dates, two-digit-year windowing, leap years, Julian conversions, pre-1970 and historical dates, daylight-saving boundaries where relevant, fractional-second precision, current-date generation, sort/range predicates and round-trip equivalence. Business-result validation remains the final control.

6. DBKEY Transformation and Traceability

A DBKEY is valuable during migration because it can identify source records and help resolve owner/member relationships. It should not automatically become the permanent relational primary key. A safer pattern is to preserve legacy identifiers in staging and cross-reference structures, resolve business or surrogate keys, validate all relationships, and retire migration-only identifiers when they are no longer required.

- Retain enough source identity to trace every target row back to the source during reconciliation.
- Do not assume physical identity is business identity.
- Validate duplicate, orphan and cardinality conditions before enabling final constraints.
- Keep cross-reference data through cutover and the agreed validation period.

7. Discovery: The Foundation of Modernization

Schema conversion without application discovery is incomplete. The database definition shows what can exist; the application shows how the data is actually navigated; production data shows what actually exists. All three views are needed.

Discovery Stream	Examples	Primary Output
Database	Schemas, subschemas, records, sets, areas, indexes	Source relationship catalog
Application	COBOL/ADS DML, CICS flows, batch, reports	Access-path and dependency inventory
Data	Counts, key distributions, set cardinalities, orphans	Data-quality baseline
Operations	Schedules, restart/recovery, SLAs, peak windows	Cutover and nonfunctional requirements
Security	Data classification, access controls, audit behavior	Target security requirements



BeyondX Principle

Discovery is not a phase - it is a discipline. Every transformation decision should be traceable to source structure, application behavior, production evidence or an approved target requirement.

8. ADS/Online: The Application Modernization Workstream

An enterprise running CA-IDMS may also rely on ADS/Online for interactive business processing. ADS/Online should be inventoried as a separate but tightly coupled modernization workstream because its dialogs, processes, maps/screens and database commands can encode business behavior that is not visible in the database schema alone.

CA-IDMS database discovery	ADS/Online / application discovery
Schemas, subschemas, areas and records	Dialogs, processes and maps/screens
Sets, owners, members and ordering	IDMS DML and navigation sequences
DBKEY, CALC, indexes and physical access	Business rules, validation and error handling
OCCURS, REDEFINES and record layouts	COBOL calls, CICS transactions and shared modules
Production cardinalities and data quality	Online transaction flows, security and external interfaces
Output: relationship catalog	Output: application dependency and business-behavior graph

8.1 Build an End-to-End Dependency Graph

The modernization team should connect each user-facing transaction to the data relationships it actually uses. A useful dependency chain is:

This graph prevents a database conversion from breaking application assumptions that were previously enforced by navigation order, set membership, status fields or dialog logic.

8.2 ADS/Online Target Choices

Objective	Candidate transformation
Remove IDMS/ADS with minimum platform change	CA-IDMS -> Db2 z/OS; ADS/Online -> COBOL/CICS or another supported mainframe model; embedded Db2 SQL
Strategic mainframe exit	CA-IDMS -> Aurora PostgreSQL; ADS/Online/COBOL logic -> Java/Spring services; web UI/API layer; cloud batch



Objective	Candidate transformation
Mixed portfolio	Transform by business domain or transaction group while maintaining controlled coexistence and reconciliation

BeyondX Principle: Modernize the database and the applications that know how to navigate it as one dependency-aware program, even when they are delivered in separate phases.

9. Three Modernization Models

Retiring CA-IDMS does not require every other technology to change at the same time, and retiring ADS/Online does not require an immediate mainframe exit. Database, application, language and infrastructure modernization are separable decisions. Three models are especially useful: a phased relational-mainframe model, an application-first hybrid model, and a direct cloud-native model. The architecture should minimize unnecessary simultaneous change while preserving a credible path to final decommissioning.

9.0.1 Application-First Hybrid Model

The hybrid model is strategically useful when an organization wants to remove ADS/Online and modernize application logic before moving the database off the mainframe. CA-IDMS is first transformed to Db2 z/OS, while ADS/Online and selected COBOL logic are refactored to Java/Spring. If the Java data-access layer uses a disciplined relational boundary - standard SQL where practical, externalized connectivity, repository/service abstractions and limited Db2-specific behavior - the later Db2 z/OS to Aurora PostgreSQL migration can be substantially easier. The Java application should largely remain intact, with remediation concentrated in connectivity, SQL dialect differences, data types, database-specific functions, transaction behavior and performance tuning.

Figure 3. Application-first hybrid modernization separates application transformation from the later database-platform exit.

BeyondX Principle: Modernize the application once. Modernize the database in controlled stages.

Path	Typical Target	What Changes	What May Remain
Path 1: Phased relational modernization	Db2 z/OS + COBOL/CICS	CA-IDMS sets/DML become relational; ADS/Online dialogs/processes are converted or refactored to a supported mainframe application model	Mainframe, COBOL/CICS, selected batch and operational controls



Path	Typical Target	What Changes	What May Remain
Path 2: Application-first hybrid modernization	Db2 z/OS + Java/Spring	CA-IDMS becomes relational on Db2; ADS/Online and selected COBOL logic move to Java/Spring behind a disciplined SQL/JDBC/service boundary	Mainframe database platform initially; Java application is positioned for a later Db2-to-Aurora PostgreSQL transition with comparatively limited application remediation
Path 3: Direct cloud-native transformation	Amazon Aurora PostgreSQL + Java/Spring on AWS	Database model, ADS/COBOL application access, runtime, batch and infrastructure	Validated business rules, outcomes and required external interfaces

Architecture Principle

Modernization is not synonymous with mainframe elimination. Database, language, application, transaction-processing and infrastructure modernization are separate engineering decisions.

9.1 Choosing Among the Three Models

Modernization objective / condition	Architecture implication
Remove CA-IDMS and ADS/Online with minimum disruption	Favor a phased Db2 z/OS + COBOL/CICS state when it materially reduces risk.
Retain the mainframe for the foreseeable future	Relational mainframe modernization may be the target state rather than an interim state.
Mainframe exit and AWS are already strategic objectives	Evaluate direct CA-IDMS -> Aurora PostgreSQL and ADS/COBOL -> Java transformation.
Transformation risk is very high or dependencies are poorly understood	Use discovery and incremental domain waves; an intermediate relational state may be justified.
Intermediate Db2 would exist only briefly	Avoid adding a second production migration unless the risk reduction clearly outweighs cost and delay.
Modernize the application now while retaining Db2 z/OS temporarily	Use the application-first hybrid model: ADS/COBOL -> Java/Spring with Db2 z/OS, while minimizing database-specific coupling so a later Aurora PostgreSQL cutover is concentrated in the data-access boundary.



An intermediate technology should be introduced only when its reduction in transformation risk justifies the additional migration, testing and reconciliation effort.

10. Industry Evolution: Claims Processing as a Modernization Pattern

Large healthcare claims-processing environments provide a useful historical pattern. Practitioner experience across several Blue Cross Blue Shield environments included evolution from legacy Claims Management System (CMS) implementations toward Automated Major Medical System (AMMS) implementations. These programs illustrate staged modernization in which CA-IDMS and Assembler dependencies could be removed or reduced while COBOL, CICS, Db2 and selected mainframe components remained in service.

This history is presented as an industry modernization pattern, not as a claim that every Blue Cross Blue Shield organization followed an identical sequence. Implementation details varied by organization and period. The engineering lesson is that database, application, language and platform modernization can be sequenced independently.

11. AWS-Native Target Architecture

For organizations whose objective includes mainframe exit and reduction of proprietary platform dependencies, Java and Amazon Aurora PostgreSQL provide a candidate AWS-native architecture. The design should remain workload-driven rather than tool-driven.

Figure 4. Reference AWS-native target architecture.

Aurora reader endpoints can distribute read-only connections across Aurora Replicas and reduce query overhead on the writer. Read/write batch activity must continue to use the writer path. RDS Proxy can improve connection management for highly elastic applications, but it should be selected based on measured connection behavior rather than treated as mandatory.

11.1 Foreign Keys and Access-Path Preservation

A foreign key preserves relational integrity, but the referencing column must also be evaluated for indexing based on query and maintenance patterns. A common IDMS owner/member traversal becomes a PostgreSQL predicate on the member table; without an appropriate index, a formerly efficient navigational path can become an expensive scan.

12. Automation and Tooling Strategy

Automation can materially accelerate inventory, code analysis, transformation, testing and deployment. It should not be represented as a substitute for semantic discovery or engineering review.

AWS Transform for mainframe currently documents support for z/OS workloads including COBOL, PL/I, JCL, CICS/BMS, Db2, IMS and VSAM, with automated refactoring toward Java and modern databases. The



published supported-source list does not currently name CA-IDMS or ADS/Online. TP-005 therefore treats CA-IDMS/ADS discovery and semantic transformation as specialized workstreams and uses AWS transformation tooling only where the documented source technologies and patterns apply.

Workstream	Automation Opportunity	Human Engineering Responsibility
Inventory	Parse source artifacts and dependencies	Confirm completeness and application boundaries
Data model	Generate candidate relational structures	Resolve business keys, cardinality, ordering and exceptions
Code	Translate/refactor supported patterns	Validate business semantics and remove procedural artifacts
Data	Automate extraction/transformation/load	Define conversion rules and reconciliation controls
Testing	Generate/replay tests and compare outputs	Approve equivalence criteria and investigate variances
Deployment	Infrastructure-as-code and CI/CD	Security, resilience, performance and cutover approval

BeyondX Principle

Tools accelerate modernization - but they do not replace engineering judgment.

13. AI Agents in CA-IDMS & ADS/Online Modernization

AI agents can materially accelerate a migration when they are grounded in the actual legacy artifacts and operate inside explicit engineering controls. In this paper, ChatGPT and OpenAI Codex are positioned prominently as AI engineering agents for reasoning across schemas, subschemas, set definitions, COBOL copybooks, ADS/Online logic, SQL, Java code, test evidence and migration documentation. OpenAI describes Codex as an agent for writing, reviewing and shipping code, including complex refactors and migrations; ChatGPT can complement that coding workflow with architectural reasoning, analysis and documentation.

The most valuable use of AI begins before code generation. An AI agent can help explain unfamiliar IDMS constructs, correlate owner/member sets with application navigation, propose relational mappings, identify DBKEY traceability requirements, analyze OCCURS and REDEFINES, draft Db2 or PostgreSQL SQL, assist with Java/Spring refactoring, generate reconciliation queries and test cases, and maintain mapping and decision documentation. These outputs remain engineering hypotheses until validated against source definitions, production data and expected business outcomes.

Figure 5. ChatGPT / OpenAI Codex as a prominent AI engineering agent within a governed modernization workflow.



Migration Stage	ChatGPT / OpenAI Codex Contribution	Required Engineering Control
Discovery	Explain IDMS schemas/sets; summarize ADS/COBOL logic; correlate dependencies; create inventories and questions for SMEs.	Validate against authoritative schema, subschema, source code and production evidence.
Data-model transformation	Propose owner/member -> PK/FK mappings; DBKEY cross-reference rules; OCCURS/REDEFINES treatments; index candidates.	Architect approves semantics, cardinality, keys, constraints and performance design.
Application transformation	Assist IDMS DML -> SQL and ADS/COBOL -> Java/Spring refactoring; review generated code.	Compile, test, code review, security review and business-rule validation.
Testing & reconciliation	Generate unit/integration tests, reconciliation SQL, orphan/cardinality checks and edge-case scenarios.	Run against controlled environments and compare to trusted baselines.
Migration operations	Draft runbooks, cutover checklists, rollback criteria, issue analysis and evidence summaries.	Production actions remain approval-gated and auditable.

BeyondX Principle: Use AI to understand the legacy estate before using AI to transform it.

AI accelerates discovery, transformation, testing and documentation - but production migration decisions require engineering validation.

14. Migration Execution Framework

1. Establish the source inventory across CA-IDMS, ADS/Online, COBOL, CICS, JCL/batch and external interfaces.
2. Build the CA-IDMS relationship catalog and connect ADS/Online dialogs/processes and COBOL programs to the records, sets and access paths they use.
3. Profile production data and establish reconciliation baselines.
4. Design the target relational model and key strategy.
5. Define legacy DBKEY cross-reference and relationship-resolution rules.
6. Transform application access from navigational DML to set-oriented SQL/services.
7. Build extraction, transformation and loading pipelines.
8. Execute iterative conversion waves in non-production environments.
9. Validate data, relationships, business outputs and performance continuously.
10. Rehearse cutover, fallback and restart procedures.
11. Cut over only after agreed acceptance criteria are satisfied.
12. Retire the legacy environment only after the validation and retention period is complete.



15. Validation Framework

Validation must prove more than row-count equality. A successful migration demonstrates that relationships, business behavior, financial or operational totals, ordering requirements and application outcomes remain correct.

Validation Level	Examples
1. Structural	Expected tables, columns, keys, constraints and datatypes
2. Record	Counts by entity, partition, status and business period
3. Key	Uniqueness, nullability and key-domain checks
4. Relationship	Owner/member cardinality, orphan detection, bridge completeness
5. Financial / aggregate	Amounts, balances, counts and control totals
6. Behavioral	Legacy navigation result vs relational query result
7. Application	Online transaction and batch output equivalence
8. Nonfunctional	Performance, concurrency, recovery, security and operational SLAs

BeyondX Principle

Validation is not a milestone - it is continuous. Reconciliation begins during discovery, continues through every conversion wave, and remains active through cutover stabilization.

16. Performance Engineering for Aurora PostgreSQL

- Design indexes from observed access paths, not from source structures alone.
- Use set-oriented SQL to replace record-at-a-time navigation wherever business behavior permits.
- Partition only where data volume, retention, pruning or operational requirements justify it.
- Separate read-only reporting or eligible batch reads to Aurora readers when consistency requirements permit.
- Keep transactional writes on the writer path and account for replica lag in read-after-write workflows.
- Size connection pools deliberately; evaluate RDS Proxy for highly elastic or connection-intensive services.
- Test representative online and batch concurrency, not isolated SQL statements only.
- Treat query plans, statistics, vacuum/autovacuum behavior and data distribution as part of production readiness.



17. Security and Operational Modernization

A cloud target changes the operational control plane as well as the database. Security architecture should include least-privilege IAM, network segmentation, encryption, secrets management, database roles, audit logging, backup and recovery, observability, vulnerability management and controlled deployment pipelines. Legacy access rules should be revalidated against current business roles rather than copied blindly.

18. Cutover, Fallback and Decommissioning

A modernization is not complete when the target receives data. It is complete when the business can operate on the target, reconciliation is accepted, recovery is proven, and the legacy platform can be retired with confidence.

Phase	Control Objective
Pre-cutover	Freeze scope, validate baselines, rehearse conversion and fallback
Cutover	Control change, execute final synchronization, validate critical transactions
Stabilization	Monitor errors, performance, data variances and business outcomes
Acceptance	Obtain formal technical and business validation
Decommission	Archive required artifacts, remove dependencies, terminate legacy cost and risk

BeyondX Principle

Modernization is successful only when the legacy system can be retired with confidence.

18.1 Decommissioning Is a Records, Knowledge and Architecture Event

Cutover and decommissioning are different decisions. A rollback backup retained for six or twelve months can support operational recovery, but it does not by itself establish records-retention compliance, preserve institutional knowledge, or document why the retired system behaved as it did. For regulated enterprises - and especially U.S. federal environments - the disposition of records, system documentation and storage media should be governed by applicable records schedules, agency policy, legal holds, security requirements and approved lifecycle procedures.

BeyondX Decommissioning Knowledge Package

Preserve	Minimum evidence before retirement
Domain knowledge	Business purpose, capabilities, terminology, rules, exceptions and authoritative



	business processes.
Stakeholders	Business owners, operational users, data stewards, downstream consumers, interfaces and external dependencies.
Data & records	Authoritative datasets, metadata, records classification, approved retention/disposition authority, legal holds and archival requirements.
Application behavior	ADS dialogs, COBOL/Java logic, calculations, workflows, batch schedules, reports, interfaces and operational dependencies.
Migration evidence	Source-to-target mappings, relationship transformations, reconciliation results, exceptions, approvals and acceptance evidence.
Retirement evidence	Final archive/retention decision, access method for retained records, shutdown approvals, sanitization/disposal evidence and decommissioning certification.

Federal records-management context

NARA guidance for electronic information systems calls for agencies to maintain inventories of their systems and the documentation needed to maintain and use electronic records. NARA records-management requirements also tie retention and disposition to approved authorities rather than to an arbitrary technical backup period. Where records must remain available after an application is retired, the modernization program should preserve the records, metadata and means needed to retrieve and interpret them for the required period. These requirements are records-management obligations; they should be applied with the agency records officer and counsel rather than treated as a database-team decision.

Security disposition is a separate control. NIST SP 800-88 Rev. 2 describes an enterprise media-sanitization program in which sanitization and disposal methods are selected and validated according to the sensitivity of the information. Retention, recoverability, knowledge preservation and sanitization therefore solve different problems and should not be collapsed into a single “keep a backup” decision.

BeyondX Principle: Decommissioning an application is not the same as disposing of its information, records, documentation or institutional knowledge.

BeyondX Principle: Backup is a recovery mechanism. Retention is a records-management obligation. Knowledge preservation is an architectural responsibility.

BeyondX Principle: A legacy system should not disappear before the organization has preserved the knowledge required to understand what disappeared.

19. BeyondX Engineering Principles for CA-IDMS & ADS/Online Modernization

- Discovery is not a phase - it is a discipline.
- The unit of modernization is the business relationship, not the legacy record.
- Preserve business meaning, not pointer mechanics.



- Do not confuse DBKEY physical identity with business identity.
- Convert navigational access into set-oriented relational behavior, not one database call per legacy verb.
- Treat database, application, language and infrastructure modernization as separable decisions.
- Use automation aggressively where it is proven, but keep engineers responsible for semantics and acceptance.
- Validation is continuous.
- Decommissioning is an explicit success criterion.

Preserve records and institutional knowledge before retirement - a rollback backup is not a records strategy.

Treat decommissioning approval, retention, archival access and sanitization as explicit lifecycle controls.

- Date/time values carry business semantics, not just bytes. Preserve their meaning, precision and temporal context before changing their representation.

Conclusion

CA-IDMS and ADS/Online environments can contain decades of business knowledge encoded across network database structures, dialog/process logic, COBOL layouts and operational procedures. Safe modernization begins by reconstructing those dependencies before choosing whether to modernize first to a relational mainframe state or transform directly to Java and Aurora PostgreSQL on AWS.

The destination technology matters, but the engineering discipline matters more. A modern target that does not preserve the correct relationships and business outcomes is not modernization. It is simply a new implementation of an old problem.

References and Technical Notes

AWS Transform - Modernization of mainframe applications

AWS documentation describes current supported z/OS workloads including COBOL, PL/I, JCL, CICS/BMS, Db2, IMS and VSAM, plus human-in-the-loop modernization workflows.

<https://docs.aws.amazon.com/transform/latest/userguide/transform-app-mainframe.html>

AWS Transform for mainframe FAQ

AWS documents automated refactoring toward Java and modern database options; the current supported-source list does not name CA-IDMS or ADS/Online.

<https://docs.aws.amazon.com/m2/latest/userguide/ba-faq.html>

Amazon Aurora Reader Endpoints

AWS documents reader endpoints for distributing read-only connections across Aurora Replicas.

<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Endpoints.Reader.html>

Amazon RDS Proxy for Aurora

AWS documents RDS Proxy connection pooling and read/write or read-only proxy endpoints for Aurora.

<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/rds-proxy-creating.html>



Broadcom CA-IDMS 19 - SQL date/time representation

CA-IDMS product documentation defines native SQL DATE, TIME and TIMESTAMP representation and external date/time formats. Exact behavior should be verified against the installed CA-IDMS release and application usage before transformation.

<https://techdocs.broadcom.com/us/en/ca-mainframe-software/database-management/ca-idms-reference/19-0/sql-reference/data-types-and-null-values/representation-of-date-time-values.html>

Broadcom CA-IDMS 19 - ADS date/time and callable services

CA-IDMS documentation describes ADS date/time functions and callable services used by legacy applications for date/time generation, conversion and display. These routines are discovery evidence; they do not imply that all network record dates share one physical representation.

<https://techdocs.broadcom.com/us/en/ca-mainframe-software/database-management/ca-idms-reference/19-0/ads-reference/ads-reference-built-in-functions/system-supplied-functions/date-functions.html>

<https://techdocs.broadcom.com/us/en/ca-mainframe-software/database-management/ca-idms-reference/19-0/callable-services-reference/idmsin01.html>

NARA - Records Inventory: Electronic Information Systems

NARA guidance states that agencies should maintain a complete and accurate inventory of electronic records systems and related documentation, consistent with 36 CFR 1236.26.

<https://www.archives.gov/records-mgmt/scheduling/inventory-systems>

NARA - Universal Electronic Records Management Requirements

NARA publishes baseline lifecycle requirements for managing electronic records, derived from statutes, regulations, standards, policy and guidance.

<https://www.archives.gov/records-mgmt/policy/universalmrequirements>

NIST SP 800-88 Rev. 2 - Guidelines for Media Sanitization

NIST provides current guidance for enterprise media sanitization and disposal based on information sensitivity.

<https://csrc.nist.gov/pubs/sp/800/88/r2/final>